



Case3



Guide de Code Review — Feuille à remplir

| Objectif : utiliser cette feuille comme un *fil conducteur* pendant l'exercice.

| Remplis les zones "👉 Réponse" au fur et à mesure.



Étape 1 — Cadrage (Top-Down / Haut Niveau)

1) Intention Métier

Question : À quoi sert ce fichier ? Quel problème résout-il ?

👉 **Réponse :**

- Contexte / produit : Pipeline d'évaluation LLM, tracing et guardrail
- Problème utilisateur / besoin : Besoin d'évaluer la perf de notre LLM (qualité, latence)
- Entrées principales : requête utilisateur
- Sorties principales : réponse générée par la demande + score llm as a judge
- Hypothèses / contraintes :

2) Architecture Globale

Question : Comment le code est-il découpé ? (classes, services, endpoints, fonctions utilitaires)

👉 **Réponse :**

- Composants / modules : Classe LLMJudgeEvaluator → définit le juge qui doit noter la réponse de 1 à 5.

- Point d'entrée (main / handler / endpoint principal) : fastapi endpoint `"/generate_with_validation"`
 - Données qui circulent (schéma rapide) : requête user (json str) → `user_query`, `generated_text` → `judge_score_str`
-

3) Périmètre & Dépendances

Question : Quels sont les modules externes importés et les flux d'entrées/sorties ?

 **Réponse :**

- Dépendances externes (librairies / services) :
 - `logging`, `typing`, `fastapi`
 - Dépendances internes (fichiers / packages du repo) :
 - `none`
 - I/O (réseau, DB, fichiers, cache, queue, etc.) :
 - requête user fastapi, appels api `mistral_client`
 - Zones sensibles (auth, secrets, PII, permissions) :
 - `nope`
-

Étape 2 — Inspection par les 4 Piliers (Scan ligne par ligne)

| Pour chaque point, note **ce que tu observes** et **les anomalies**.

| (Tu peux référencer des lignes/fonctions : `foo()`, `MyClass.method`, "ligne ~120", etc.)

Pilier 1 — Logique & Concurrency

Checklist mentale :

- Le flux d'exécution est-il correct ?
- Async / threads / événements : pas de blocage, pas de race conditions, pas d'await oubliés ?

 **Notes / anomalies :**

- audit_quality_synchronous → Ilm as a judge synchrone → ajoute latency au critical path → mettre en background/queue (?)
 -
 -
-

Pilier 2 — Gestion des Ressources & Mémoire

Checklist mentale :

- Pas de fuite d'état global, objets bien isolés ?
- Connexions / fichiers / sockets : cycle de vie propre (ouverture/fermeture garanties) ?

Notes / anomalies :

- -
 -
-

Pilier 3 — Robustesse & Sécurité

Checklist mentale :

- Erreurs / exceptions : traitées correctement (loggées, relancées, pas masquées) ?
- Inputs : validés (null/absent/format), pas d'injection, pas de confiance implicite ?

Notes / anomalies :

- for item in test_dataset: → pas de try except, si crash au revoir
 - request: Request → vérifier pydantic
-

Pilier 4 — Architecture, Design & Performance

Checklist mentale :

- SRP : une seule responsabilité par fonction/classe ?
- Traitements lourds inutiles (boucles, N+1, requêtes multiples, allocations) ?
- Nommage clair, pas ambigu ?

Notes / anomalies :

- temperature=0.7 dans un benchmark → non reproductibilité → potentiel echec du CI/CD + pas de seed (g pas compris ca représente quoi le seed mais oklm)
- # 1. Tracing de l'étape initiale → mais plus de tracing après ! générer un uuid, et faire du tracing de time dans un span pour chq chose time intensive
- judge_prompt = f"Évalue la pertinence de cette réponse sur une échelle de 1 à 5 :\nQuestion: {user_query}\nRéponse: {generated_answer}" → evaluation nulle et pas objective → mettre un pydantic avec des champs bool

Étape 3 — Feedback Structuré (SÉVÉRITÉ / IMPACT / SOLUTION)

A) Rappel des tags de sévérité

- **[BLOCKER]** : crash fatal, bug logique critique, faille de sécurité majeure
- **[MAJOR]** : perf sous charge, fuite de ressources, mauvaise architecture
- **[MINOR]** : lisibilité, typage manquant, refactoring local
- **[NIT]** : cosmétique / style

B) Commentaires de review (à remplir)

Copie-colle ce bloc autant de fois que nécessaire :

- [TAG_SÉVÉRITÉ] Emplacement / Fonction :
 - Problème :
 - Impact (en prod) :
 - Solution (proposition concrète, idéalement avec pseudo-code / code) :

Commentaire 1 :

- [MAJOR] : audit_quality_synchronous
 - Problème : llm as a judge synchrone dans le critical path
 - Impact : rajoute de la latence inutile a l'utilisateur
 - Solution : mettre le process en async background/queue

Commentaire 2 :

- [MAJOR] evals:
 - Problème : par de try / except aux appels llm benchmark
 - Impact : si un crash, on veut quand même continuer
 - Solution : mettre un try / except + logger erreur

Commentaire 3 :

- [BLOCKER] request: Request :
 - Problème : pas de vérification pydantic de la requête
 - Impact : si pas les bons champs crash
 - Solution : classe pydantic. Mais en fait jsuis pas sur. Est ce que y'a vraiment besoin de vérifier? Psk la requête arrivante elle est pas formulée par le LLM c'est l'utilisateur après j'pense c bien formaté de manière déterministe?

Commentaire 4 :

- [BLOCKER] benchmark:
 - Problème : temperature=0.7 dans un benchmark
 - Impact : non reproductibilité des test de performance -> le CI/CD qui conduit les test automatiques va détecter une baisse de perf et annuler. Pareil pour la seed mais j'ai pas compris ça sert à quoi
 - Solution : utiliser temp=0.0, seed=42 fixed

Commentaire 5 :

- [BLOCKER] tracing:
 - Problème : génération d'un tracer au début mais jamais utilisé
 - Impact : on ne peut pas avoir la trace de cet appel llm, problème de tracabilité en cas de bug / autre jsp
 - Solution : générer un uuid à l'appel, puis le transmettre à chaque appel et chaque tâche intensive créer un span vers opentelemetry je crois? qui va mesurer la latence

Commentaire 6 :

- [____] judge_prompt :
 - Problème : Évalue la pertinence de cette réponse sur une échelle de 1 à 5
 - Impact : pas d'info objective / quantitative / comparable, impossible d'évaluer proprement avec ça.
 - Solution : Utiliser des champs bool pydantic